30 AI Agent Interview Questions & Answers

The Complete Guide to Crack AI Agent Interviews in 2026

First-Principles Thinking | Easy Language | Real Examples

AI Agents	**MCP Protocol**	**A2A Protocol**
Tool Calling	**Agent Memory**	**RAG vs Agents**
Multi-Agent Systems	**LangGraph**	**CrewAI**
	Agent Evaluation	

AmanAI Lab

amanailab.com | YouTube: @AmanAILab

FREE RESOURCE - Share with your network!

Table of Contents

Q23. Explain the core concepts of LangGraph: State, Nodes, and Edges.

Q24. What is Human-in-the-Loop in LangGraph and why is it important?

■ CrewAI

Q25. What is CrewAI and how does it simplify building multi-agent systems?

Q26. What are the different Process types in CrewAI?

Q27. How does CrewAI compare to LangGraph for building agents?

■ Agent Evaluation

Q28. How do you evaluate an AI Agent's performance?

Q29. What is Agent Benchmarking and what are popular benchmarks?

Q30. How do you test agents in production and handle failures?

AI Agents Fundamentals

Q1 What is an AI Agent, and how is it different from a regular chatbot?

Answer:

An AI Agent is an autonomous system that can **perceive its environment, reason about it, make decisions, and take actions** to achieve a goal - all with minimal human intervention. A chatbot simply takes input and returns output (one turn at a time). An agent can plan multi-step workflows, use external tools, remember past interactions, and self-correct when things go wrong.

■ Example:

```
Chatbot: You ask 'Book me a flight' -> it says 'I can't do that.'
AI Agent: You ask 'Book me a flight' -> it checks your calendar, searches
flights, compares prices, picks the best option, and books it - all
autonomously.
```

■ *Interview Tip: Always mention the 3 pillars - Perception, Reasoning, Action. That's what separates an agent from a simple LLM wrapper.*

Q2 What is the ReAct (Reasoning + Acting) pattern in AI Agents?

Answer:

ReAct is a prompting framework where the agent alternates between **Thought** (reasoning about what to do), **Action** (calling a tool or API), and **Observation** (reading the result). This loop continues until the agent reaches a final answer. It combines chain-of-thought reasoning with tool use, making the agent both a thinker and a doer.

■ Example:

```
User: 'What is the weather in Tokyo and should I carry an umbrella?'
Thought: I need to check Tokyo weather first.
Action: call_weather_api('Tokyo')
Observation: 28C, 80% chance of rain
Thought: High rain probability, I should recommend umbrella.
Final Answer: 'It is 28C in Tokyo with 80% chance of rain. Yes, carry an
umbrella.'
```

■ *Interview Tip: Draw the Thought -> Action -> Observation loop on a whiteboard. Interviewers love visual explanations.*

Q3 What is the difference between Reactive and Proactive agents?

Answer:

Reactive agents respond only when triggered - they wait for input and then act. **Proactive agents** can initiate actions on their own based on goals, schedules, or environment changes. Most production agents today are a hybrid - they react to user input but can also proactively monitor, alert, and execute tasks autonomously.

■ **Example:**

```
Reactive: A customer support agent that answers only when a user sends a
message.
Proactive: A monitoring agent that detects your server CPU is at 95%,
automatically scales up resources, and sends you a Slack alert - all without you
asking.
```

■ *Interview Tip: Mention that proactive agents need a 'goal loop' or 'event-driven architecture' to keep running in the background.*

MCP (Model Context Protocol)

Q4 What is MCP (Model Context Protocol) and why does it matter?

Answer:

MCP is an **open protocol created by Anthropic** that standardizes how AI models connect to external tools, data sources, and services. Think of it as the 'USB-C for AI' - before MCP, every tool integration needed custom code. MCP provides a universal interface so any AI model can talk to any tool using a standard format. It follows a client-server architecture where the AI app is the client and each tool/service runs as an MCP server.

■ **Example:**

```
Without MCP: You write custom Python code to connect Claude to Google Drive,
then different code for Slack, then different code for GitHub - 3 different
integrations.
With MCP: Each service exposes an MCP server. Claude connects to all of them
using the same protocol. Add a new tool? Just plug in another MCP server.
```

■ *Interview Tip: Compare MCP to REST APIs - REST standardized web communication, MCP standardizes AI-tool communication.*

Q5 Explain the MCP architecture - what are Hosts, Clients, and Servers?

Answer:

Host: The AI application the user interacts with (e.g., Claude Desktop, an IDE). **Client**: Lives inside the host; maintains a 1:1 connection with an MCP server. **Server**: A lightweight program that exposes specific tools, resources, or prompts via MCP. The host can have multiple clients, each connected to a different server. Communication uses JSON-RPC 2.0 over stdio (local) or HTTP with SSE (remote).

■ **Example:**

```
Host: Claude Desktop app
Client 1 -> MCP Server: Google Drive (read/write files)
Client 2 -> MCP Server: GitHub (create PRs, read issues)
Client 3 -> MCP Server: Slack (send messages, read channels)

Each server independently exposes its capabilities. The host orchestrates which
server to call.
```

■ *Interview Tip: Drawing this 3-layer diagram (Host -> Clients -> Servers) instantly shows you understand the architecture.*

Q6 What are the three core primitives that an MCP Server can expose?

Answer:

An MCP server can expose three types of capabilities: (1) **Tools** - functions the model can call (like search_files, send_email). These are model-controlled. (2) **Resources** - data the model can read (like file contents, database records). These are application-controlled. (3) **Prompts** - reusable prompt templates for specific workflows. These are user-controlled. This separation gives fine-grained control over what the AI can do vs. what it can read vs. what templates it can use.

■ **Example:**

```
MCP Server for a database:
Tool: run_query(sql) - executes a SQL query
Resource: schema://tables - exposes table schemas for context
Prompt: 'analyze_data' - a pre-built prompt template for data analysis

The model picks which tool to call, the app decides which resources to load, and
the user selects prompts.
```

■ *Interview Tip: Remember T-R-P (Tools, Resources, Prompts) and who controls each (Model, App, User).*

A2A (Agent-to-Agent Protocol)

Q7 What is A2A (Agent-to-Agent Protocol) and how is it different from MCP?

Answer:

A2A is an **open protocol by Google** that enables AI agents to communicate, collaborate, and delegate tasks to each other - regardless of which framework or vendor built them. While MCP connects a model to tools (vertical integration), A2A connects agent to agent (horizontal integration). Think of it this way: MCP gives an agent its hands (to use tools), A2A gives agents the ability to talk to each other and collaborate.

■ Example:

```
MCP scenario: Agent calls a weather API tool to get data.
A2A scenario: A travel-planning agent delegates flight search to a specialized
flight-booking agent, hotel search to a hotel agent, and combines their results.

MCP = Agent <-> Tool
A2A = Agent <-> Agent
```

■ *Interview Tip: Say 'MCP is vertical (model-to-tool), A2A is horizontal (agent-to-agent). They are complementary, not competing.'*

Q8 What is an Agent Card in A2A?

Answer:

An Agent Card is a **JSON metadata file** (hosted at /.well-known/agent.json) that describes what an agent can do. It includes the agent's name, description, supported skills, endpoint URL, and authentication requirements. When one agent wants to discover and collaborate with another, it first reads the Agent Card to understand the other agent's capabilities - like a business card for AI agents.

■ Example:

```
{
"name": "FlightBooker",
"description": "Books flights and searches for deals",
"skills": ["search_flights", "book_ticket", "cancel_booking"],
"endpoint": "https://flights.example.com/a2a",
"auth": {"type": "oauth2"}
}

A travel planner agent reads this card and knows it can delegate flight tasks to
FlightBooker.
```

■ *Interview Tip: Compare Agent Cards to OpenAPI/Swagger specs - same concept but for agent capabilities instead of REST endpoints.*

Q9 What is a Task in A2A and what are its lifecycle states?

Answer:

A Task is the **core unit of work** in A2A. When a client agent sends a request to a remote agent, it creates a Task. The task goes through defined states: **submitted** (request sent), **working** (agent is processing), **input-required** (agent needs more info from client), **completed** (done successfully), **failed** (error occurred), **canceled** (stopped by client). This lifecycle allows long-running, asynchronous workflows where agents can take minutes or hours to complete complex tasks.

■ Example:

```
Client Agent: 'Book cheapest flight NYC to London next Friday'
Task state: submitted -> working (searching flights)
Task state: working -> input-required ('Window or aisle seat?')
Client responds: 'Window'
Task state: working -> completed (Booking confirmed: BA117, $450, Window seat)
```

■ *Interview Tip: Emphasize that 'input-required' makes A2A conversational, not just fire-and-forget.*

Tool Calling

Q10 What is Tool Calling (Function Calling) in LLMs and how does it work?

Answer:

Tool calling is the ability of an LLM to **decide when to use an external function** and generate the correct arguments for it. The LLM does NOT execute the function itself - it outputs a structured JSON specifying which function to call and with what parameters. Your application code then executes the function and feeds the result back to the LLM. This closes the loop: LLM reasons -> picks tool -> app executes -> result goes back -> LLM generates final answer.

■ Example:

```
System: You have tool get_stock_price(ticker: str)
User: 'What is Apple stock price?'
LLM output: {"tool": "get_stock_price", "args": {"ticker": "AAPL"}}
App executes: get_stock_price('AAPL') -> returns $198.50
Fed back to LLM -> 'Apple (AAPL) stock is currently at $198.50.'
```

■ *Interview Tip: Stress that the LLM only DECIDES which tool to call. It never executes anything - your code does. This is a common misconception.*

Q11 How do you handle errors and hallucinated tool calls?

Answer:

There are 3 main issues: (1) **Hallucinated tools** - LLM invents a tool that doesn't exist. Fix: validate tool name against your registered list before executing. (2) **Wrong arguments** - correct tool but bad parameters. Fix: use JSON schema validation (like Pydantic) to verify args. (3) **Execution errors** - tool fails at runtime. Fix: catch exceptions, send error message back to LLM, let it retry or try alternative tool. Always implement a **max retry limit** to prevent infinite loops.

■ Example:

```
LLM says: call 'search_internet(query=xyz)'
But your tools only have: search_web, get_weather, send_email

Step 1: 'search_internet' not in tool list -> reject
Step 2: Return to LLM: 'Tool not found. Available tools: search_web,
get_weather, send_email'
Step 3: LLM self-corrects: call 'search_web(query=xyz)' -> works!
```

■ *Interview Tip: Mention 'guardrails' - input validation, output validation, retry limits, and fallback strategies.*

Q12 What is Parallel Tool Calling and when would you use it?

Answer:

Parallel tool calling is when the LLM requests **multiple tool calls in a single response**, and your application executes them simultaneously instead of sequentially. This is useful when tools are independent of each other. It reduces latency significantly. For example, if the agent needs weather AND stock price AND news - these are independent calls that can run in parallel. However, if tool B depends on the output of tool A, they must be sequential.

■ Example:

```
User: 'Give me a morning briefing'
LLM output (single response):
tool_call_1: get_weather('Mumbai')
tool_call_2: get_stock_portfolio()
tool_call_3: get_top_news(category='tech')

All 3 execute in parallel using asyncio.gather()
Total time: ~max(individual times) instead of sum(individual times)
```

■ *Interview Tip: Mention asyncio.gather() in Python or Promise.all() in JS as implementation patterns for parallel execution.*

Memory in Agents

Q13 What are the different types of memory in AI Agents?

Answer:

AI agents typically use 4 types of memory: (1) **Short-term / Working Memory** - the current conversation context window. Lost when session ends. (2) **Long-term Memory** - persistent storage across sessions (vector DB, database). Survives restarts. (3) **Episodic Memory** - remembers specific past interactions and experiences ('Last time you asked about Python, I recommended Flask'). (4) **Semantic Memory** - stores general knowledge and facts the agent has learned over time. In practice, most agents combine short-term (context window) + long-term (vector DB) memory.

■ Example:

```
Short-term: User says 'my name is Aman' -> agent remembers within this chat.
Long-term: Next day, user returns -> agent retrieves from DB: 'Welcome back,
Aman!'
Episodic: 'Last week you asked about RAG pipelines. Want to continue?'
Semantic: Agent knows 'Python is a programming language' as general knowledge.
```

■ *Interview Tip: Map these to human memory - short-term is like working memory (RAM), long-term is like hard disk, episodic is like personal diary.*

Q14 How do you implement long-term memory in an AI Agent?

Answer:

The most common approach: (1) After each conversation, **extract key facts/summaries** from the dialogue. (2) **Generate embeddings** of these summaries using an embedding model. (3) **Store in a vector database** (Pinecone, ChromaDB, Weaviate). (4) On new conversations, **retrieve relevant memories** by embedding the current query and doing similarity search. (5) **Inject retrieved memories** into the system prompt. You can also use a structured database (PostgreSQL) for exact facts like user preferences, and vector DB for semantic/fuzzy memories.

■ Example:

```
Conversation 1: User discusses Python FastAPI project.
-> Extract: 'User is building a FastAPI app with Azure SQL backend'
-> Embed and store in ChromaDB

Conversation 2 (days later): User asks 'help me fix my API'
-> Embed query, search ChromaDB
-> Retrieved memory: 'User has a FastAPI + Azure SQL project'
-> Agent responds with context: 'Sure! Is this for your FastAPI Azure project?'
```

■ *Interview Tip: Mention the trade-off - vector DB for semantic search, relational DB for structured facts. Best agents use both.*

Q15 What is the 'memory overflow' problem and how do you solve it?

Answer:

LLMs have a **finite context window** (e.g., 128K tokens). As conversations grow longer and more memories accumulate, you can't fit everything in the prompt. This is the memory overflow problem. Solutions: (1) **Summarization** - periodically compress older messages into summaries. (2) **Relevance filtering** - only retrieve memories relevant to the current query using similarity search. (3) **Sliding window** - keep last N messages verbatim, summarize the rest. (4) **Tiered memory** - hot (recent, in context), warm (summarized), cold (archived in DB, retrieved on demand).

■ Example:

```
100-message conversation with a 4K token limit:

Messages 1-70: Summarized into 1 paragraph ('User discussed ML pipeline
architecture...')
Messages 71-95: Summarized into 3 bullet points
Messages 96-100: Kept verbatim (full detail)

This 'sliding window + summarization' fits within the token limit while
preserving context.
```

■ *Interview Tip: Compare to database indexing - you don't load all rows in memory, you query what you need. Same principle.*

RAG vs Agents

Q16 What is RAG and how is it fundamentally different from AI Agents?

Answer:

RAG (Retrieval-Augmented Generation) is a pattern where you retrieve relevant documents from a knowledge base and feed them to an LLM to generate grounded answers. It is a **one-shot pipeline**: retrieve -> augment prompt -> generate. An **AI Agent**, on the other hand, can plan multi-step actions, use multiple tools (including RAG), make decisions, and iterate. RAG is a specific technique; an agent is an autonomous system. An agent might USE RAG as one of its many tools.

■ Example:

```
RAG: User asks 'What is our refund policy?' -> retrieve policy doc -> LLM
generates answer. Done.

Agent: User asks 'Process a refund for order #123' -> Agent step 1: look up
order in database -> step 2: check refund policy (uses RAG) -> step 3: calculate
refund amount -> step 4: initiate refund via payment API -> step 5: send
confirmation email. Five steps, multiple tools, autonomous execution.
```

■ *Interview Tip: Say 'RAG is like a librarian that finds books. An agent is like a manager that finds books AND takes actions based on what they say.'*

Q17 When should you use RAG vs an Agent vs both together?

Answer:

Use **RAG alone** when: task is pure Q&A; over documents, no actions needed, low latency required. Use an **Agent alone** when: task requires actions (booking, sending, updating), multi-step reasoning, tool orchestration. Use **both together (Agentic RAG)** when: agent needs to answer questions from knowledge bases AND take actions. In Agentic RAG, the agent decides when to search, which knowledge base to query, evaluates result quality, and can re-search with different queries if results are poor.

■ Example:

```
RAG only: Internal FAQ bot for company policies.
Agent only: Automated CI/CD pipeline that tests, builds, and deploys code.
Agentic RAG: Customer support agent that searches product docs (RAG), checks
order status (tool), processes returns (tool), and escalates to human if needed
(decision).
```

■ *Interview Tip: Agentic RAG is the hottest architecture in production right now. Mention it proactively to score bonus points.*

Q18 What is Agentic RAG and how is it different from basic RAG?

Answer:

In basic RAG, the retrieval pipeline is **fixed** - same query, same retriever, same top-K, one shot. In Agentic RAG, an **agent controls the retrieval process**: it decides WHEN to retrieve, WHAT query to use, which retriever/index to search, evaluates whether results are good enough, and can re-retrieve with a modified query. The agent adds a reasoning layer on top of RAG. It can also route queries to different knowledge bases, combine results from multiple sources, and fall back to web search if internal docs don't have the answer.

■ Example:

```
Basic RAG: query -> embed -> search Pinecone -> top 5 chunks -> LLM answer.

Agentic RAG:
Agent thinks: 'This question is about billing'
-> Routes to billing_knowledge_base (not general KB)
-> Retrieves 5 chunks
-> Agent evaluates: 'These chunks are about old pricing, let me search again'
-> Re-queries with refined terms
-> Gets better results -> generates answer
```

■ *Interview Tip: The key word is 'adaptive retrieval' - the agent adapts its search strategy based on results.*

Multi-Agent Systems

Q19 What are Multi-Agent Systems and why are they useful?

Answer:

A Multi-Agent System (MAS) is an architecture where **multiple specialized agents collaborate** to solve complex tasks. Instead of one super-agent doing everything, you break the problem into sub-tasks and assign each to a specialist agent. Benefits: (1) **Separation of concerns** - each agent is expert in one domain. (2) **Scalability** - add new agents without changing existing ones. (3) **Reliability** - if one agent fails, others continue. (4) **Parallel execution** - independent agents work simultaneously.

■ Example:

```
Task: 'Write a research report on quantum computing'

Agent 1 (Researcher): Searches web, gathers information
Agent 2 (Writer): Takes research, writes the report
Agent 3 (Editor): Reviews for quality, grammar, factual accuracy
Agent 4 (Formatter): Converts to PDF with proper formatting

Each agent is good at ONE thing. Together they produce better results than one
generalist agent.
```

■ *Interview Tip: Compare to microservices architecture in software - same principle of specialized, independent, communicating components.*

Q20 What are the common communication patterns in Multi-Agent Systems?

Answer:

Four main patterns: (1) **Sequential / Pipeline** - Agent A passes output to Agent B, then to C. Like an assembly line. (2) **Hierarchical** - A manager/orchestrator agent delegates tasks to worker agents and combines results. (3) **Peer-to-peer / Collaborative** - Agents communicate directly with each other, debating or building on each other's work. (4) **Broadcast** - One agent publishes information, all others receive it. The right pattern depends on the task: sequential for workflows, hierarchical for complex orchestration, peer-to-peer for creative/debate tasks.

■ Example:

```
Sequential: Researcher -> Writer -> Editor (report writing)
Hierarchical: Manager assigns 'search flights' to FlightAgent, 'search hotels'
to HotelAgent, combines both for travel plan.
Peer-to-peer: Two agents debate pros and cons of a business strategy, reaching
consensus.
Broadcast: Market data agent publishes price update, all trading agents receive
it simultaneously.
```

■ *Interview Tip: Always mention which pattern you'd pick for a given scenario and WHY. That shows design thinking.*

Q21 How do you handle conflicts when multiple agents disagree?

Answer:

Conflict resolution strategies: (1) **Voting / Majority** - multiple agents vote, majority wins. Good for classification tasks. (2) **Authority hierarchy** - a supervisor agent has final say. Good for production systems. (3) **Debate + Judge** - agents present arguments, a separate judge agent decides. Good for quality. (4) **Confidence-based** - agent with highest confidence score wins. (5) **Human-in-the-loop** - escalate to a human when agents can't agree. Best practice: always have a fallback (usually human escalation) for high-stakes decisions.

■ Example:

```
Task: 'Should we approve this loan?'

Agent A (Risk): 'REJECT - high debt ratio'
Agent B (Revenue): 'APPROVE - good repayment history'
Agent C (Compliance): 'APPROVE - meets all regulatory requirements'

Voting: 2-1 APPROVE.
Authority: Risk agent has veto power -> REJECT.
Judge: Supervisor agent reviews all reasoning -> conditional APPROVE with lower
limit.
```

■ *Interview Tip: Mention that the conflict resolution strategy should match the risk level of the decision.*

LangGraph

Q22 What is LangGraph and how is it different from LangChain?

Answer:

LangChain is a framework for building LLM-powered applications using chains (linear sequences) of components. **LangGraph** is built on top of LangChain and adds **graph-based orchestration** with cycles, branching, and state management. The key difference: LangChain chains are DAGs (no loops). LangGraph allows cycles - meaning an agent can loop back, retry, and iterate. This makes LangGraph ideal for building agents that need complex, iterative workflows with conditional routing.

■ **Example:**

```
LangChain: prompt -> LLM -> output parser -> done (linear, no going back)

LangGraph:
start -> LLM reasons -> should I use a tool?
-> YES -> call tool -> check result -> good enough?
-> NO -> loop back to LLM with error -> retry
-> YES -> generate final answer -> end

The cycles (loops) are what make LangGraph powerful for agents.
```

■ *Interview Tip: Say 'LangChain is for chains, LangGraph is for agents' - that one line shows you get the distinction.*

Q23 Explain the core concepts of LangGraph: State, Nodes, and Edges.

Answer:

State: A shared dictionary/object that flows through the graph. Every node can read and update it. This is how information passes between steps. **Nodes**: Functions that do the actual work - call an LLM, execute a tool, process data. Each node takes the current state, does something, and returns an updated state. **Edges**: Connections between nodes. Can be fixed (always go A to B) or **conditional** (go to B if condition X, else go to C). Conditional edges are what enable branching and looping.

■ Example:

```
State: {"query": "...", "messages": [], "tool_result": None, "final_answer":
None}

Nodes:
'agent' node: runs LLM, decides next action
'tool' node: executes the selected tool
'answer' node: formats final response

Edges:
'agent' --[needs tool]--> 'tool'
'agent' --[has answer]--> 'answer'
'tool' --> 'agent' (always loop back to agent after tool)
'answer' --> END
```

■ *Interview Tip: If asked to 'build an agent', describe it as a LangGraph with state, at least 2 nodes, and one conditional edge.*

Q24 What is Human-in-the-Loop in LangGraph and why is it important?

Answer:

Human-in-the-Loop (HITL) in LangGraph means the agent **pauses execution at a specific node** and waits for human approval, input, or correction before continuing. LangGraph implements this via **checkpointing** - it saves the full graph state, waits for human input, then resumes from exactly where it paused. This is critical for production agents where you can't let AI make high-stakes decisions alone (financial transactions, data deletion, sending emails to clients).

■ **Example:**

```
Agent graph flow:
1. Research node: gathers data (automatic)
2. Draft node: writes email to client (automatic)
3. HUMAN CHECKPOINT: 'Here is the draft email. Approve / Edit / Reject?'
-> Human approves -> continue
-> Human edits -> update state with edits -> continue
-> Human rejects -> loop back to draft node
4. Send node: sends approved email
```

■ *Interview Tip: Always recommend HITL for actions that are irreversible (sending emails, payments, deletions). This shows production-readiness thinking.*

CrewAI

Q25 What is CrewAI and how does it simplify building multi-agent systems?

Answer:

CrewAI is a Python framework for **orchestrating role-playing autonomous AI agents**. It simplifies multi-agent systems by letting you define: **Agents** (with role, goal, backstory), **Tasks** (what to accomplish), and **Crew** (a team of agents working together). Instead of manually coding communication between agents, CrewAI handles delegation, collaboration, and task execution automatically. It's like assembling a team where each member has a clear job description.

■ Example:

```
from crewai import Agent, Task, Crew

researcher = Agent(role='Researcher', goal='Find latest AI trends',
backstory='Expert in AI research')
writer = Agent(role='Writer', goal='Write engaging blog post',
backstory='Experienced tech blogger')

task1 = Task(description='Research AI trends 2026', agent=researcher)
task2 = Task(description='Write blog from research', agent=writer)

crew = Crew(agents=[researcher, writer], tasks=[task1, task2])
result = crew.kickoff()
```

■ *Interview Tip: Compare CrewAI to a real company - agents are employees with roles, tasks are assignments, crew is the team.*

Q26 What are the different Process types in CrewAI?

Answer:

CrewAI supports different process types that control HOW agents work together: (1) **Sequential** - tasks execute one after another, each agent's output becomes the next agent's input. Simple and predictable. (2) **Hierarchical** - a manager agent automatically delegates tasks to worker agents, reviews results, and orchestrates the workflow. The manager decides task order and reassignment. (3) **Consensual** (experimental) - agents collaborate as equals, discussing and reaching agreement.

■ Example:

```
Sequential: Researcher -> Writer -> Editor
(Assembly line - fixed order)

Hierarchical:
Manager agent receives 'write a report'
-> Delegates research to Researcher agent
-> Reviews research, delegates writing to Writer agent
-> Reviews draft, asks Editor to polish
-> Manager compiles final output

The hierarchical process is best for complex projects where task order isn't
predetermined.
```

■ *Interview Tip: Say 'Sequential for predictable pipelines, Hierarchical when you need dynamic task routing' - shows you know when to use which.*

Q27 How does CrewAI compare to LangGraph for building agents?

Answer:

CrewAI: Higher-level abstraction. Define agents with roles, give them tasks, and the framework handles orchestration. Best for: quick prototyping, role-based multi-agent workflows, when you want simplicity. **LangGraph**: Lower-level, graph-based. You define every node, edge, and state transition. Best for: custom control flows, complex conditional logic, production systems needing fine-grained state management. Think of it as: CrewAI is like Django (batteries-included), LangGraph is like Flask (flexible, build-your-own).

■ Example:

```
Same task: Build a content creation pipeline

CrewAI (10 lines): Define 3 agents with roles, 3 tasks, 1 crew. Done.
LangGraph (50+ lines): Define state schema, create nodes for each step, add
conditional edges, handle state updates manually.

CrewAI is faster to build. LangGraph gives more control.
Production recommendation: Prototype in CrewAI, migrate to LangGraph if you need
custom control flow.
```

■ *Interview Tip: Don't say one is 'better' - say each fits a different use case. That shows engineering maturity.*

Agent Evaluation

Q28 How do you evaluate an AI Agent's performance?

Answer:

Agent evaluation has multiple dimensions: (1) **Task Success Rate** - did the agent complete the task correctly? (2) **Step Efficiency** - how many steps/tool calls did it take? Fewer is better. (3) **Tool Selection Accuracy** - did it pick the right tools? (4) **Reasoning Quality** - were intermediate thinking steps logical? (5) **Latency** - total time to complete the task. (6) **Cost** - total tokens/API calls consumed. (7) **Error Recovery** - when something went wrong, did it self-correct? You evaluate each dimension separately because an agent can succeed at the task but be inefficient, or fail the task but show good reasoning.

■ Example:

```
Agent task: 'Find and summarize the top 3 AI papers this week'

Task Success: Correct papers found? 3/3 = 100%
Step Efficiency: Used 5 tool calls (optimal was 3) = 60%
Tool Selection: Called search_arxiv correctly = 100%
Reasoning: Logical chain of thought = Good
Latency: 45 seconds (benchmark: 30s) = Acceptable
Cost: 15K tokens ($0.03) = Within budget
Error Recovery: Handled one API timeout gracefully = Good
```

■ *Interview Tip: Create a 'scorecard' approach - list these dimensions as a table. Interviewers love structured evaluation frameworks.*

Q29 What is Agent Benchmarking and what are popular benchmarks?

Answer:

Agent benchmarking involves **testing agents on standardized tasks** to compare performance across models, frameworks, and architectures. Popular benchmarks: (1) **SWE-bench** - tests if agents can solve real GitHub issues (software engineering). (2) **WebArena** - tests web navigation and task completion. (3) **GAIA** - tests general AI assistant capabilities with real-world questions. (4) **AgentBench** - tests agents across operating systems, databases, web browsing. (5) **HumanEval** - tests code generation. These benchmarks tell you how well your agent performs compared to others on the same standardized tasks.

■ **Example:**

```
You build a coding agent using GPT-4 + LangGraph.
You run it on SWE-bench: solves 38% of GitHub issues.
Competitor uses Claude + CrewAI: solves 42%.

SWE-bench gives you an apples-to-apples comparison.

You then identify WHERE your agent fails (e.g., multi-file edits) and improve
that specific capability.
```

■ *Interview Tip: Mention SWE-bench by name - it's the most talked-about agent benchmark in 2026. Shows you're current.*

Q30 How do you test agents in production and handle failures?

Answer:

Production agent testing strategy: (1) **Unit tests** - test individual tools and nodes in isolation. (2) **Integration tests** - test full agent workflows with mock APIs. (3) **Trajectory testing** - verify the agent takes the expected sequence of steps (not just final output). (4) **Shadow mode** - run agent in parallel with human operators, compare outputs without affecting real users. (5) **Canary deployment** - roll out to 5% of traffic first, monitor metrics. (6) **Observability** - log every agent step, tool call, decision point for debugging. For failure handling: implement circuit breakers (stop after N failures), fallback to simpler agents or human handoff, and always have a graceful degradation path.

■ Example:

```
Production pipeline:
1. Unit test: Does search_tool return valid JSON?
2. Integration: Full booking flow with mock flight API.
3. Trajectory: Agent should call search -> filter -> book (not book -> search).
4. Shadow: Run agent alongside human support team for 1 week.
5. Canary: 5% real users -> monitor success rate.
6. Full rollout: 100% users with circuit breaker (if 3 failures in 5 min ->
pause agent, alert team).
```

■ *Interview Tip: Mention 'trajectory testing' specifically - most candidates only talk about output testing. Testing the path (not just destination) shows advanced thinking.*

You Made It! ■

You've just gone through 30 real AI Agent interview questions with first-principles explanations and practical examples.

■ Subscribe to AmanAI Lab on YouTube for video explanations

■ Download the 50 ML Interview Questions PDF (companion resource)

■ Share this PDF with someone preparing for AI interviews

■ Follow on LinkedIn for daily AI/ML tips

amanailab.com

Built with ❤ by AmanAI Lab | 2026